

GRACE: A GraphRAG Causal Benchmark

Welcome to the **GRACE** benchmark repository. This project aims to evaluate the robustness and causal reasoning capabilities of various GraphRAG (Retrieval-Augmented Generation over Knowledge Graphs) frameworks when faced with different graph perturbations.

Installation & Setup

This project uses modern Python dependency management via **uv**.

```
# Cài đặt nền tảng và môi trường ảo
uv sync
```

Configuration (Environment Variables)

Before running the benchmark, you must configure the LLM endpoints. We use a **.env** file for easy management.

```
# Copy the provided template
cp .env.template .env
```

Open the **.env** file and insert your configuration for local LLM and remote Embedding API:

- **OPENAI_API_KEY**: Required for Nvidia Embeddings.
- **OPENAI_BASE_URL**: Point to Nvidia's API (<https://integrate.api.nvidia.com/v1>).
- **EMBEDDING_BASE_URL**: Point to Nvidia's API (<https://integrate.api.nvidia.com/v1>).
- **EMBEDDING_MODEL_NAME**: Set the embedding model (default: **nvidia/nv-embed-v1**).
- **LLM_BASE_URL**: Point to your local vLLM instance (e.g., <http://localhost:8001/v1>).
- **LLM_MODEL_NAME**: Set your local model name (e.g., **cyankiwi/Qwen3-30B-A3B-Instruct-2507-AWQ-4bit**).

Running Benchmarks

1. Start Local vLLM Server

We run the 30 tỷ tham số LLM locally (**cyankiwi/Qwen3-30B-A3B-Instruct-2507-AWQ-4bit**) using 4-bit bitsandbytes quantization to fit into a single GPU (e.g., RTX 5880 Ada with 48GB VRAM).

First, install the required serving dependencies if you haven't already:

```
uv pip install vllm "bitsandbytes>=0.48.1"
```

Start the vLLM server on a dedicated Terminal. Dưới đây là giải thích chi tiết cho các cấu hình (config) đang được sử dụng để chạy model này mượt mà:

- **CUDA_VISIBLE_DEVICES=0**: Chỉ định phiên chạy trên GPU số 0 (chọn GPU có VRAM trống đủ để load 30B model).
- **VLLM_USE_V1=0**: Tắt Engine V1 mới của vLLM để dùng lại V0. Engine V1 hiện tại đang lỗi tương thích bộ nhớ với dòng model MoE (Mixture of Experts) to như Qwen.
- **--quantization compressed-tensors**: Quá trình nhồi model vào RAM. Model do cộng đồng đóng gói được định dạng dưới chuẩn **compressed-tensors**, nên vLLM cần cấu hình này để phân giải model 4-bit thành công.
- **--max-model-len 16384**: Mở rộng độ dài Context Window từ 8K lên 16K tokens. Điều này cực kỳ quan trọng để đảm bảo thuật toán Graph Retrieval của LightRAG không bị tràn context (trả về lỗi **None**) khi truy xuất các Subgraphs quá lớn và có chiều dài.
- **--gpu-memory-utilization 0.9**: Cho quyền vLLM được chiếm đến 90% thẻ GPU để tăng tỉ lệ cache nội bộ KV.
- **--enforce-eager**: Ép buộc chạy dạng Eager thay cho CUDA Graph. Với model MoE, CUDA Graph hay gặp lỗi fragmentation (kẹt phân mảnh bộ nhớ) gây dừng startup.

```
CUDA_VISIBLE_DEVICES=2 vllm serve cyankiwi/Qwen3-30B-A3B-Instruct-2507-AWQ-4bit \
  --host 0.0.0.0 \
  --port 8001 \
  --max-model-len 32768 \
  --gpu-memory-utilization 0.9
```

*Note: Loading and quantizing a 60GB+ 30B model on-the-fly may take 10-15 minutes. Wait until you see **Uvicorn running on http://0.0.0.0:8001** before proceeding.*

2. Data Preparation

Before evaluating, you must prepare the benchmark dataset by fetching Wikidata metadata, extracting clean subgraphs, generating perturbations, and converting to ABCD Multiple Choice format.

Run the complete data preparation pipeline:

```
uv run python scripts/data_generation/prepare_benchmark_data.py
```

This script will:

1. Fetch metadata (labels, descriptions, aliases) for all Wikidata entities (**data/wikidata_labels.json**)
2. Extract clean test subgraphs from LC-QuAD 2.0 (**data/test_clean_subgraphs/**)
3. Generate causal perturbations (broken, swapping, topological, type_matching variants) (**data/test_perturbed_subgraphs/**)
4. Convert all datasets to ABCD Multiple Choice Question format with fixed distractors

Note: The benchmark now uses **ABCD Multiple Choice Questions** instead of open-ended QA. Each question has 4 options:

- **A:** Ground Truth answer
- **B:** Swapped Entity (for swapping variant) or random distractor
- **C:** "Insufficient information to answer" (for broken graphs)
- **D:** Random distractor

The correct answer varies by perturbation type:

- **clean, topological, type_matching:** Correct answer is **A** (Ground Truth)
- **broken:** Correct answer is **C** (Insufficient info)
- **swapping:** Correct answer is **B** (Swapped entity)

All options remain fixed across variants for the same question to ensure fair comparison.

3. Centralized Prompts

To ensure fair evaluation across all RAG frameworks, we use a single, strict benchmark instruction. This guarantees that models output **pure entity names** and handle dataset anomalies correctly (e.g., avoiding literal numbers when the dataset ground truth expects an entity, or isolated/broken graphs).

This prompt is located at: [src/graphrag_benchmark/prompts.py](#)

4. Benchmark Evaluation

Current benchmark framework list:

1. LightRAG
2. ...

4.1 LightRAG

4.1.1 Benchmark Execution

After your local vLLM server is running and data is prepared, open a **new Terminal** to run the complete causal benchmark evaluation script.

Run across all subgraphs:

```
uv run scripts/benchmark/lightrag/run_all_lightrag.py
```

Run a single query by ID:

```
uv run python scripts/benchmark/lightrag/benchmark_lightrag.py 2466
```

Note: The script will load a subgraph scenario from `.json` files, construct Knowledge Graphs dynamically using the RAG engine for all available variants (`clean`, `broken`, `type_matching`, `topological`, `swapping`), query the local Qwen LLM while fetching Embeddings from Nvidia. Results are scored by extracting the ABCD choice from model output and comparing against the correct letter for each variant. Results are saved to `data/lightrag/benchmark_results.json`.

4.1.2 Metrics

After running the benchmark, evaluate the results using the metrics script:

```
uv run scripts/analysis/metrics_evaluation.py
```

This computes **Multi-Class Classification Metrics** (Accuracy, Macro-F1) across all variants:

- Each variant (clean, broken, etc.) is evaluated independently
- Metrics measure how well the RAG model selects the correct ABCD letter
- Results help diagnose if perturbations affect causal reasoning robustness

You can also evaluate custom result files:

```
uv run scripts/analysis/metrics_evaluation.py --results_file  
data/lightrag/benchmark_results.json
```

4.1.3 Cleanup Failed Predictions (`None`) & Temporary Progress Stats

When a run is interrupted or some samples fail (model output/predicted letter is `None`), you can clean the current results file to keep only successful questions, then rerun the benchmark for missing ones.

This utility is framework-agnostic and can be used for any benchmark result JSON that follows the same schema (`question_id`, `variants`, `predicted_letter`, `mcq_correct_letter`).

Run cleanup + report:

```
uv run python scripts/eval/cleanup_and_report_results.py  
data/lightrag/benchmark_results.json
```

Generic usage (other frameworks):

```
uv run python scripts/eval/cleanup_and_report_results.py  
data/<framework>/benchmark_results.json
```

What this script does:

- Remove questions that contain any failed variant (`predicted_letter` is `None`/null/empty, or `answer` is `None`/`ERROR`).
- Overwrite the same input JSON file you pass to the script.
- Save a timestamp backup automatically before overwrite, for example:
`data/lightrag/benchmark_results.json.bak_YYYYMMDD_HHMMSS`.
- Print temporary progress: successful questions / total questions.
- Print per-mode accuracy using only valid predictions (ignoring `None`/error rows).

If you do not want to create a backup file:

```
uv run python scripts/eval/cleanup_and_report_results.py
data/lightrag/benchmark_results.json --no-backup
```

After cleanup, run benchmark again:

```
uv run scripts/benchmark/lightrag/run_all_lightrag.py
```

Data Extraction & Analysis

The data pipeline scripts have been logically classified within the `scripts/` folder:

- `scripts/data_generation/`: Utilities for extracting and perturbing source data (LC-QuAD 2.0).
- `scripts/analysis/`: Diagnostic utilities to verify metadata, logs, and diagram visualization.
- `scripts/benchmark/lightrag/`: LightRAG-specific benchmark execution and evaluation scripts.

If you want to learn how the datasets are exactly extracted from Wikidata, how causal perturbations are applied, and how they are statistically analyzed or visualized, please read our detailed methodology guide in:

 [README_DATASET.md](#)